



KCA UNIVERSITY

School of Computing and Information Technology
Department of Computing

SYSTEM DESIGN SPECIFICATION (SDS)

Predicting Developer Technology Decline Using Community Signals and Enterprise Adoption Patterns: A Machine Learning Approach (TechPulse)

Unit Code & Title: STU 4101 – Final Year Project I

Program: Bachelor of Science in Data Science

Student Name: Kitili Tony Kenga

Registration Number: 24/03652

Supervisor: Dr. Rufus Gireka

ORCID: 0009-0007-6899-8590

Date Submitted: 10th July 2026

Faculty: School of Technology (SoT)

Contents

3.1 Introduction.....	4
3.1.1 Purpose.....	4
3.1.2 Scope.....	4
3.1.3 System Overview	4
3.1.4 Definitions, Acronyms and Abbreviations.....	4
3.2 System Overview	5
3.2.1 System Context	5
3.2.2 Assumptions and Dependencies.....	5
3.3 System Architecture	5
3.3.1 High-Level Architecture Diagram	5
3.3.2 Modules and Components.....	6
3.3.3 Technology Stack.....	7
Frontend Technologies.....	7
Backend Technologies	7
Programming Languages	7
Frameworks.....	7
Database Management System (DBMS)	7
APIs.....	7
Third-party Services.....	7
Development Tools	7
3.4 Database Design.....	7
3.4.1 Data Model / Database Schema	7
3.4.2 Data Flow Diagrams (DFDs)	8
3.4.3 Data Storage.....	9
3.4.4 Data Security and Privacy.....	9
3.5 Program Design	10
3.5.1 Module Design.....	10
Module 1 — Data Ingestion & Validation.....	10
Module 2 — Feature Engineering.....	10
Module 3 — Model Training & Evaluation.....	11
Module 4 — Explainability (SHAP).....	11
Module 5 — Prediction Store & Dashboard Rendering	12
3.6 Interface Design	12
3.6.1 User Interface (UI)	13
Login Screen	13

Dashboard	13
Main Functional Screens.....	13
Navigation Flow.....	13
Wireframes / Mockups.....	14
3.6.2 External System Interfaces.....	14
APIs.....	14
Payment Gateways.....	14
Authentication Services	14
Other Third-party Systems.....	14
3.7 Deployment Design	14
3.7.1 Infrastructure Diagram.....	14
3.7.2 Deployment Strategy	15
Deployment Environment.....	15
Manual/Automated Deployment.....	15
Rollback Strategy	15
Hosting Environment	15
3.8 Security Design.....	16
3.8.1 Authentication and Authorization.....	16
3.8.2 Data Protection.....	16
3.8.3 Audit and Logging	16
References.....	17
Books	17
Research Papers	17
Documentation.....	17
Framework Documentation	17
APIs	17
Standards.....	17
Other Technical References	17
Appendix A: Full Database Schema	18
Appendix B: Flowcharts	18
Appendix C: Wireframes	18
Appendix D: UML Diagrams	19
Appendix E: API Specifications	19
Appendix F: Configuration Files	19

CHAPTER THREE: SYSTEM DESIGN SPECIFICATION

3.1 Introduction

3.1.1 Purpose

This System Design Specification (SDS) translates the objectives, requirements, and methodology defined in the TechPulse Project Proposal into a concrete technical design. It describes the architecture, database, program modules, interfaces, deployment strategy, and security controls required to build a supervised machine learning system that classifies software technologies as growing, stable, or declining, and that surfaces those predictions through an explainable, publicly accessible dashboard. The document is intended to guide implementation during Semester 1 and Semester 2 of the project and to serve as a technical reference for the supervisor, examiners, and any future contributor to the developer-ecosystem-analytics repository.

3.1.2 Scope

The design covers the extension of the existing developer-ecosystem-analytics repository with a predictive layer. It includes: the feature-engineering pipeline that derives predictive variables from six existing datasets; four supervised classification models (Logistic Regression, K-Nearest Neighbours, Random Forest, and XGBoost); a SHAP-based explainability engine; a PostgreSQL data and prediction store; and a Streamlit dashboard deployed on Streamlit Cloud. The design explicitly excludes new primary data collection, financial/investment advisory logic, real-time (sub-batch) monitoring, and mobile-native clients, consistent with Section 1.6 of the proposal.

3.1.3 System Overview

TechPulse is a batch-oriented, explainable machine learning system layered on top of an existing descriptive analytics repository. Raw signals — Stack Overflow community activity, Fortune 500 enterprise adoption depth, and developer sentiment survey data — are transformed into engineered features, used to train and evaluate four classification models, and the best-performing model's predictions are explained using SHAP and rendered on an interactive public dashboard. The system is designed to be re-run on a scheduled batch basis rather than continuously, reflecting the update cadence of its underlying data sources (annual developer surveys, periodic Fortune 500 stock snapshots).

3.1.4 Definitions, Acronyms and Abbreviations

Term	Definition
SDS	System Design Specification
ML	Machine Learning
SHAP	SHapley Additive exPlanations — a model-agnostic feature-attribution technique
KNN	K-Nearest Neighbours classification algorithm

Term	Definition
XGBoost	Extreme Gradient Boosting — a regularised gradient-boosted tree algorithm
DBMS	Database Management System
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
API	Application Programming Interface
CI	Continuous Integration
ROC-AUC	Receiver Operating Characteristic – Area Under Curve
Trajectory Label	The predicted class of a technology: growing, stable, or declining

3.2 System Overview

3.2.1 System Context

TechPulse sits between two existing boundaries: (1) the six read-only datasets already integrated into the developer-ecosystem-analytics PostgreSQL database, and (2) the end users — individual developers and organisational technology decision-makers — who consume trajectory predictions through a browser. TechPulse does not own or modify the source datasets; it reads from them, derives features, trains models offline, and writes predictions to a dedicated prediction store that the dashboard queries at request time. The full architecture diagram is presented in Figure 5.1 (Section 5.3.1).

3.2.2 Assumptions and Dependencies

- The six existing datasets (Stack Overflow questions, developer sentiment survey, Fortune 500 tech stacks, company profiles, technology metadata, question–company mapping) remain accessible in the existing PostgreSQL 14+ instance throughout the project.
- Historical data volume is sufficient to construct a labelled training set of growing / stable / declining technologies; technologies with insufficient history will be flagged as low-confidence rather than excluded outright.
- Streamlit Community Cloud free-tier hosting remains available for the duration of the project; a local fallback build is maintained as a mitigation (see Appendix D of the proposal).
- The project depends on Python 3.10+, scikit-learn, XGBoost, SHAP, and Streamlit remaining compatible with one another; version pins will be recorded in a requirements.txt / lock file.
- No new personally identifiable data is collected; all datasets are already anonymised/aggregated at the technology or company level.

3.3 System Architecture

3.3.1 High-Level Architecture Diagram

TechPulse follows a four-layer, batch-pipeline architecture: a Data Sources Layer (the six existing datasets), a Data & Feature Layer (PostgreSQL plus the feature-engineering module), a Modelling Layer (training, evaluation, and SHAP explainability), and an Application/Presentation Layer (the Streamlit dashboard). Each layer communicates through well-defined tables or artefacts rather than direct in-memory coupling, which allows the modelling layer to be re-run independently of the dashboard.

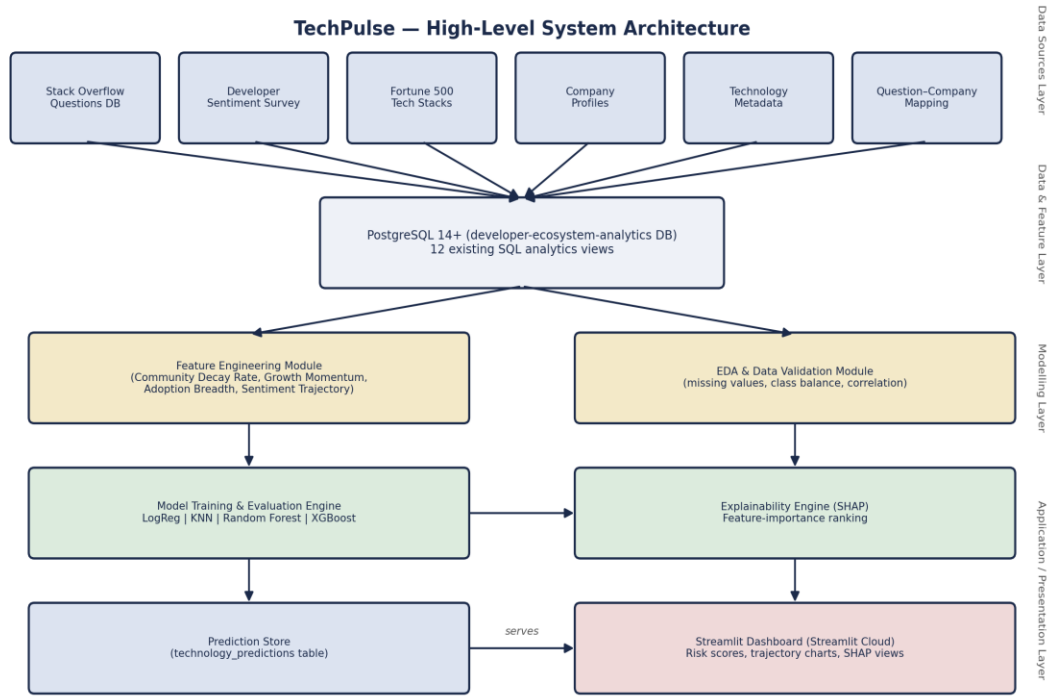


Figure 5.1 — TechPulse High-Level Architecture Diagram

3.3.2 Modules and Components

#	Module	Responsibility
1	Data Ingestion & Validation	Connects to PostgreSQL, extracts the six datasets, validates completeness and class balance
2	Feature Engineering	Derives Community Decay Rate, Growth Momentum Index, Adoption Breadth, and Sentiment Trajectory features
3	Model Training & Evaluation	Trains and cross-validates Logistic Regression, KNN, Random Forest, and XGBoost classifiers
4	Explainability (SHAP)	Computes SHAP values for the best-performing model and ranks feature importance
5	Prediction Store	Persists trajectory labels, risk scores, and SHAP summaries to PostgreSQL
6	Dashboard / Presentation	Streamlit application rendering predictions, trends, and explanations to end users

3.3.3 Technology Stack

Frontend Technologies

- Streamlit (Python-based reactive web framework) for the dashboard UI
- Plotly and Altair (via Streamlit) for interactive charts

Backend Technologies

- Python 3.10+ batch-processing scripts and scheduled jobs for ETL, feature engineering, and model training

Programming Languages

- Python (data processing, modelling, dashboard) and SQL (analytics queries, schema definitions)

Frameworks

- scikit-learn (Logistic Regression, KNN, Random Forest, pipeline utilities)
- XGBoost (gradient-boosted tree classifier)
- SHAP (model explainability)
- Streamlit (dashboard framework)

Database Management System (DBMS)

- PostgreSQL 14+ — reused from the existing developer-ecosystem-analytics repository

APIs

- Internal read/write data-access layer (SQLAlchemy) exposing feature and prediction queries to the dashboard
- No external paid APIs are required; all data sources are already integrated into the existing database

Third-party Services

- Streamlit Community Cloud for public dashboard hosting
- GitHub for version control and CI

Development Tools

- Jupyter Notebooks for reproducible EDA and experimentation
- Git/GitHub for version control; GitHub Actions for CI (linting, tests, scheduled retraining)
- VS Code as the primary IDE

3.4 Database Design

3.4.1 Data Model / Database Schema

TechPulse extends the existing schema with two new tables — `feature_matrix` and `technology_predictions` — while reusing the five existing entity tables (technologies,

stackoverflow_metrics, sentiment_survey, fortune500_stacks, companies, question_company_map) as read-only sources. The entity relationship diagram below summarises the design; the full DDL is provided in Appendix A.



Figure 5.2 — Entity Relationship Diagram for TechPulse

3.4.2 Data Flow Diagrams (DFDs)

The Level-0 DFD below traces data from the external sources, through extraction and validation, feature engineering, model training/scoring, and finally to the published dashboard consumed by end users.

Level-0 Data Flow Diagram — TechPulse

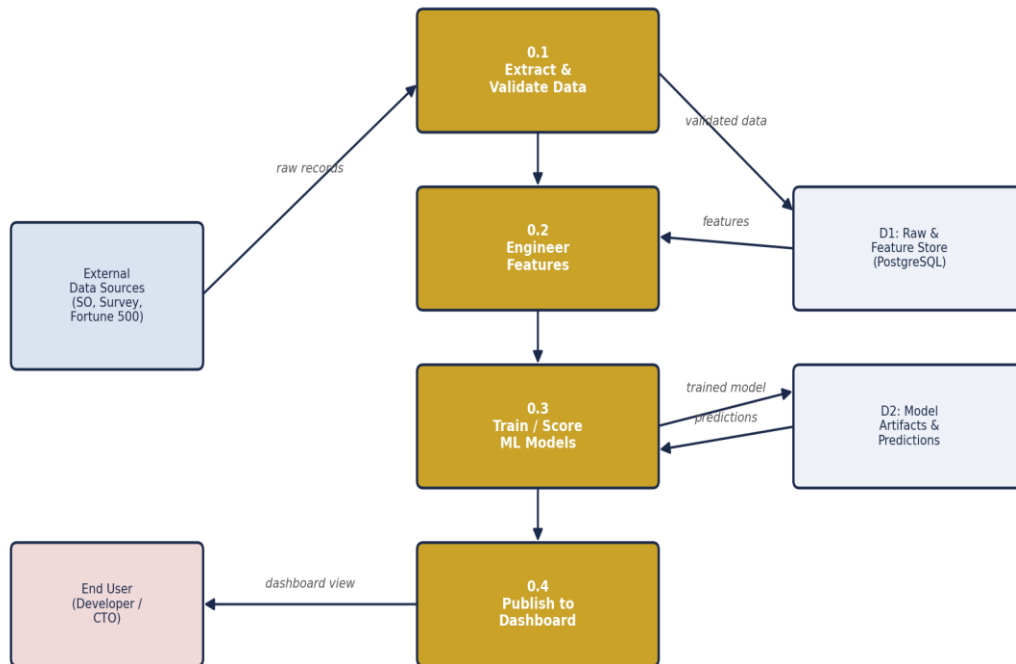


Figure 5.3 — Level-0 Data Flow Diagram

3.4.3 Data Storage

- Raw and descriptive data remain in the existing PostgreSQL tables; no source data is duplicated unnecessarily.
- The `feature_matrix` table stores one row per technology per computation run, versioned by a `computed_at` timestamp, enabling reproducibility and rollback.
- Trained model artefacts (pickled scikit-learn / XGBoost objects) are stored on disk / object storage and referenced by a `model_version` identifier in `technology_predictions`, rather than stored inside the relational database.
- The `technology_predictions` table stores the `trajectory_label`, a numeric `risk_score` (0–1), and a JSON column summarising the top SHAP features for that prediction, so the dashboard can render explanations without recomputing SHAP at request time.

3.4.4 Data Security and Privacy

- All datasets are aggregated at the technology or company level; no individual developer personal data is stored or exposed.
- Database credentials are stored as environment variables / GitHub Secrets, never committed to source control.
- Read-only database roles are used for the dashboard's query layer, while write access (feature/prediction population) is restricted to the scheduled batch job's service account.
- Connections to PostgreSQL use TLS/SSL where supported by the hosting provider.

3.5 Program Design

3.5.1 Module Design

This section presents the detailed process design for each functional module identified in Section 5.3.2, including inputs, outputs, processing logic, and pseudocode.

Module 1 — Data Ingestion & Validation

Description: Connects to the existing PostgreSQL database, extracts the six source datasets, and validates completeness, data types, and class distribution before any downstream processing begins.

Inputs: Database connection credentials; the six source tables (technologies, stackoverflow_metrics, sentiment_survey, fortune500_stacks, companies, question_company_map).

Outputs: Validated, in-memory DataFrames per dataset; a data-quality report flagging missing values, duplicate records, and class imbalance.

Processing Logic: The module opens a connection using SQLAlchemy, executes parameterised extraction queries per dataset, checks schema conformity, computes summary statistics, and raises a validation warning if any technology lacks a minimum data threshold (as per the proposal's risk mitigation in Appendix D).

```
BEGIN
connection <- OPEN_DB_CONNECTION(credentials)
FOR each dataset IN [technologies, stackoverflow_metrics, sentiment_survey,
                    fortune500_stacks, companies, question_company_map]:
    df <- EXECUTE_QUERY(connection, dataset.extract_sql)
    REPORT <- VALIDATE(df) // nulls, dtypes, duplicate keys
    IF REPORT.has_critical_issues:
        LOG_WARNING(dataset, REPORT)
    STORE df IN memory_cache[dataset.name]
END FOR
class_balance <- COMPUTE_CLASS_DISTRIBUTION(memory_cache['technologies'])
IF any_class_count < MIN_THRESHOLD:
    FLAG technology AS low_confidence
RETURN memory_cache, validation_report
END
```

Module 2 — Feature Engineering

Description: Derives the predictive features used by all four classifiers from the validated datasets, reusing existing SQL analytics views where possible (e.g., Technology Health Score) and computing new trend-based features.

Inputs: Validated DataFrames from Module 1 (Stack Overflow metrics, sentiment survey, Fortune 500 stacks).

Outputs: A single feature_matrix table/DataFrame with one row per technology, ready for model training.

Processing Logic: For each technology, the module computes a Community Decay Rate (rate of change in question volume and engagement over a rolling window), a Growth Momentum Index (recent vs historical activity ratio), Adoption Breadth (count and depth of Fortune 500 companies using the

technology), and a Sentiment Trajectory Score (change in satisfaction/adoption-intent across consecutive survey years).

```
BEGIN
FOR each technology IN technologies:
  so_series <- FILTER(stackoverflow_metrics, tech_id = technology.id)
  decay_rate <- COMPUTE_TREND(so_series.question_volume, window = 12_months)
  momentum <- (RECENT_AVG(so_series, 3_months) / HISTORICAL_AVG(so_series, 24_months))
  adoption_breadth <- COUNT(DISTINCT fortune500_stacks.company_id
    WHERE tech_id = technology.id)
  sentiment_scores <- FILTER(sentiment_survey, tech_id = technology.id)
  sentiment_trend <- LINEAR_SLOPE(sentiment_scores.satisfaction BY year)
  feature_row <- { tech_id: technology.id, decay_rate, momentum,
    adoption_breadth, sentiment_trend }
  APPEND feature_row TO feature_matrix
END FOR
PERSIST feature_matrix TO feature_matrix TABLE (versioned by computed_at)
RETURN feature_matrix
END
```

Module 3 — Model Training & Evaluation

Description: Trains and cross-validates the four candidate classifiers on the engineered feature matrix, using stratified sampling to handle class imbalance, and selects the best-performing model for production scoring.

Inputs: feature_matrix (from Module 2) joined with the target trajectory_label derived from historical technology outcomes.

Outputs: Four trained model artefacts, a comparative evaluation report (accuracy, macro-F1, ROC-AUC, confusion matrices), and a selected best_model reference.

Processing Logic: The module performs an 80/20 stratified train-test split, applies 5-fold cross-validation to each of the four algorithms, records per-class precision/recall/F1 and ROC-AUC, and persists the model with the highest macro-F1 score as the production model.

```
BEGIN
X, y <- SPLIT_FEATURES_AND_LABEL(feature_matrix)
X_train, X_test, y_train, y_test <- STRATIFIED_SPLIT(X, y, test_size = 0.2)
models <- [LogisticRegression(), KNeighborsClassifier(),
  RandomForestClassifier(), XGBClassifier()]
results <- []
FOR each model IN models:
  scores <- CROSS_VALIDATE(model, X_train, y_train, folds = 5)
  model.FIT(X_train, y_train)
  preds <- model.PREDICT(X_test)
  metrics <- EVALUATE(y_test, preds) // accuracy, macro-F1, ROC-AUC
  APPEND (model, metrics) TO results
END FOR
best_model <- ARGMAX(results BY macro_F1)
PERSIST best_model TO model_store (versioned)
RETURN results, best_model
END
```

Module 4 — Explainability (SHAP)

Description: Computes SHAP values for the selected best-performing model to identify and rank the strongest predictors of technology decline, producing both global and per-technology explanations.

Inputs: best_model and X_test / X_full from Module 3.

Outputs: Global feature-importance ranking; per-technology top-N contributing features, stored as JSON for dashboard rendering.

Processing Logic: An appropriate SHAP explainer (TreeExplainer for Random Forest/XGBoost, KernelExplainer/LinearExplainer for Logistic Regression/KNN) is selected based on the chosen best_model, SHAP values are computed for every technology, and the top contributing features per row are extracted and summarised.

```
BEGIN
explainer <- SELECT_EXPLAINER(best_model) // Tree, Linear or Kernel
shap_values <- explainer.COMPUTE(X_full)
global_importance <- MEAN(ABS(shap_values), axis = features)
FOR each technology IN X_full:
  top_features <- TOP_N(shap_values[technology], n = 5)
  STORE top_features AS JSON IN technology_predictions.shap_top_features
END FOR
RETURN global_importance, per_technology_explanations
END
```

Module 5 — Prediction Store & Dashboard Rendering

Description: Persists the final trajectory predictions and risk scores to the database and renders them, together with SHAP explanations, on the interactive Streamlit dashboard.

Inputs: best_model predictions, risk scores, and SHAP summaries from Modules 3 and 4.

Outputs: Populated technology_predictions table; rendered dashboard views (Overview, Technology Detail, SHAP Insights).

Processing Logic: Predictions are written to PostgreSQL in a single batch transaction; the Streamlit application queries this table on each user session (cached with a time-to-live) and renders KPI cards, trend charts, a sortable risk table, and a SHAP panel for the technology selected by the user.

```
BEGIN
FOR each technology IN best_model.predictions:
  risk_score <- NORMALISE(technology.predicted_probability)
  INSERT INTO technology_predictions
    (tech_id, model_name, trajectory_label, risk_score,
     shap_top_features, predicted_at)
END FOR
COMMIT TRANSACTION

// Dashboard request handling (per user session)
ON dashboard_load:
  predictions <- QUERY(technology_predictions, CACHE_TTL = 1_hour)
  RENDER KPI_cards(predictions)
  RENDER trend_chart(predictions, selected_tech)
  RENDER risk_table(predictions, filters)
  RENDER shap_panel(predictions[selected_tech].shap_top_features)
END
```

3.6 Interface Design

3.6.1 User Interface (UI)

Login Screen

A lightweight administrative login screen restricts access to data-refresh and retraining controls (public visitors browse the read-only dashboard without authentication). It presents username/email and password fields and a Sign In action.

Dashboard

The main dashboard presents KPI summary cards (counts of growing/stable/declining technologies and average risk), a trajectory trend chart, a sortable/filterable risk-score table, and a SHAP feature-importance panel for the technology currently selected.

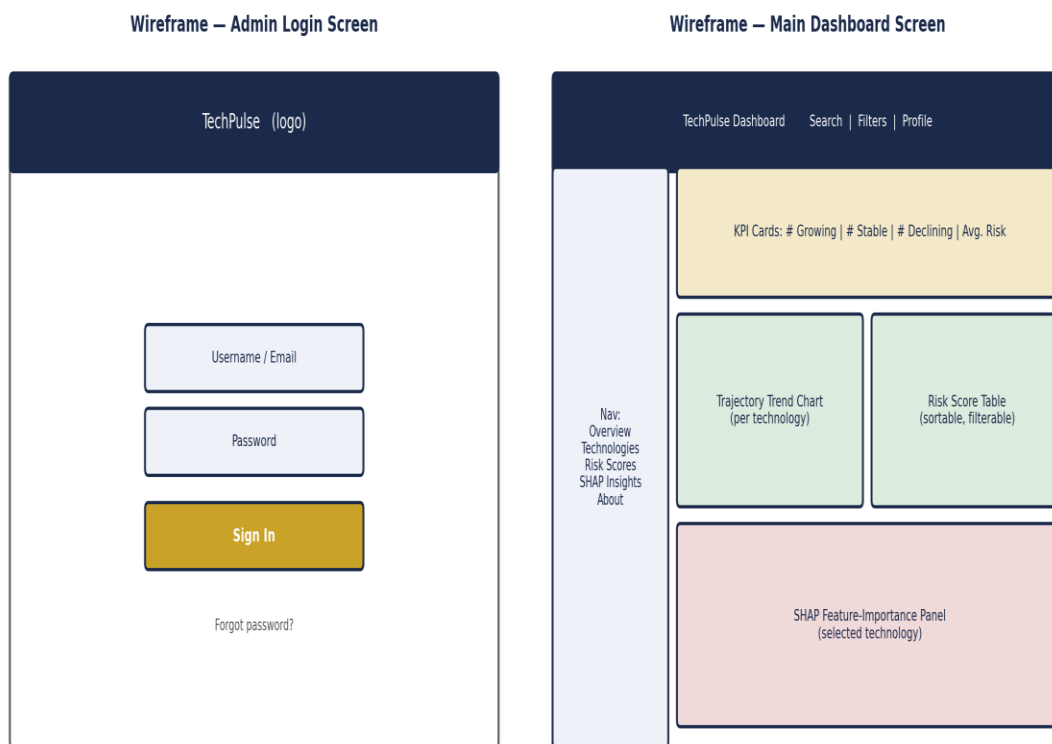


Figure 5.4 — Wireframes: Admin Login Screen and Main Dashboard Screen

Main Functional Screens

- Overview screen — aggregate KPIs and distribution of trajectory labels across all technologies
- Technology Detail screen — per-technology trend history, risk score, and SHAP explanation
- Risk Table screen — sortable, filterable table of all technologies by risk score and category
- About / Methodology screen — description of data sources, models, and evaluation metrics for transparency

Navigation Flow

Users land on the Overview screen, from which they can navigate to the Risk Table, filter or search for a specific technology, and drill into its Technology Detail screen to view SHAP-based explanations. A persistent left-hand navigation panel provides access to all screens from anywhere in the application.

Wireframes / Mockups

Low-fidelity wireframes for the login and dashboard screens are provided in Figure 5.4 above and reproduced at full size in Appendix C.

3.6.2 External System Interfaces

APIs

TechPulse does not depend on external third-party APIs for its core prediction functionality; all required data already resides in the existing PostgreSQL database. An internal data-access API (a thin SQLAlchemy-based Python layer) mediates all reads/writes between the modelling pipeline, the prediction store, and the dashboard.

Payment Gateways

Not applicable. TechPulse is a free, publicly accessible academic tool with no monetisation or payment functionality.

Authentication Services

Administrative access (data refresh, retraining triggers) uses a simple username/password login backed by hashed credentials stored in PostgreSQL; no third-party identity provider is required for this academic-scope deployment.

Other Third-party Systems

- GitHub — source control and CI/CD triggers for scheduled retraining
- Streamlit Community Cloud — application hosting for the public dashboard

3.7 Deployment Design

3.7.1 Infrastructure Diagram

TechPulse — Deployment / Infrastructure Diagram

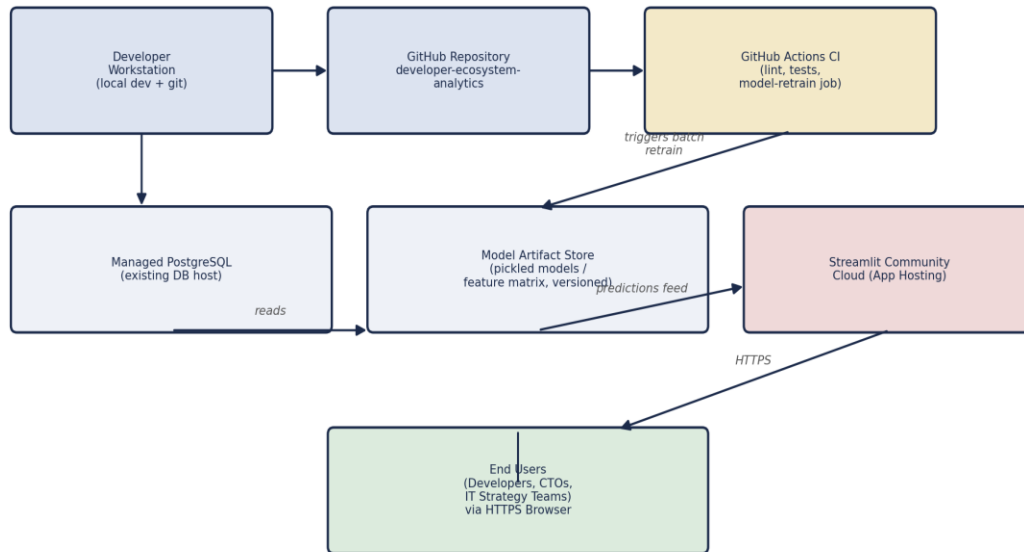


Figure 5.5 — TechPulse Deployment / Infrastructure Diagram

3.7.2 Deployment Strategy

Deployment Environment

Development occurs locally on the student's workstation (8GB+ RAM, mid-range CPU, per Appendix A of the proposal), with code pushed to the existing GitHub repository. The production dashboard is deployed to Streamlit Community Cloud, which pulls directly from the GitHub repository.

Manual/Automated Deployment

Application deployment to Streamlit Cloud is automated on every push to the main branch (Streamlit Cloud auto-redeploys on repository changes). Model retraining is triggered on a scheduled batch basis via a GitHub Actions workflow rather than on every code push, reflecting the periodic (not real-time) nature of the underlying data.

Rollback Strategy

Model artefacts are versioned (model_version identifier); if a newly trained model underperforms on the held-out evaluation set, the previous production model_version remains active and the prediction store is not overwritten until the new model passes the evaluation gate defined in Section 3.5 of the proposal. Application code rollback uses standard Git revert / redeploy of a previous commit.

Hosting Environment

- Application hosting: Streamlit Community Cloud (free tier)

- Database hosting: existing managed PostgreSQL 14+ instance used by developer-ecosystem-analytics
- Fallback: a local Streamlit build documented for offline evaluation/demo if cloud hosting is unavailable

3.8 Security Design

3.8.1 Authentication and Authorization

- Public dashboard views are read-only and require no authentication, consistent with the project's open-source, community-benefit goal.
- Administrative functions (triggering retraining, editing feature definitions) require authenticated login with hashed (bcrypt) password storage.
- Role-based database access separates a read-only dashboard role from a read-write batch-job service account.

3.8.2 Data Protection

- All secrets (database credentials, deployment tokens) are stored as environment variables / GitHub Secrets and excluded from version control via .gitignore.
- Database connections use TLS/SSL encryption in transit where supported by the hosting provider.
- Because all source datasets are aggregated at technology/company level, no personally identifiable developer data is processed, reducing privacy risk.

3.8.3 Audit and Logging

- Every batch pipeline run (ingestion, feature engineering, training, SHAP computation, publishing) logs start/end timestamps, row counts, and validation warnings to a run_log table for traceability.
- Model evaluation metrics for every training run are retained (not overwritten) to allow historical comparison and to detect performance drift over time.
- GitHub Actions CI logs provide an audit trail of code changes, test results, and deployment events.

References

Books

None used specifically for this design specification; system design draws on standard software engineering and data-engineering practice.

Research Papers

- Borges, H. & Valente, M. T. (2018). What's in a GitHub star? Understanding repository starring practices in a social coding platform. *Journal of Systems and Software*, 146, 112–129.
- Mens, T. & Goeminne, M. (2012). Analysing the evolution of social coding platforms. *Science of Computer Programming*, 78(8), 1034–1053.
- Ray, B., Posnett, D., Filkov, V., & Devanbu, P. (2014). A large scale study of programming languages and code quality in GitHub. *Proceedings of FSE 2014*, 155–165.
- Syer, M. D., Adams, B., Hassan, A. E., & Godfrey, M. W. (2013). Revisiting prior empirical findings for agile: An analysis of the RapidSMS open source project. *Proceedings of CASCON 2013*, 49–62.

Documentation

- PostgreSQL 14 Documentation — <https://www.postgresql.org/docs/14/>
- Python 3.10 Documentation — <https://docs.python.org/3.10/>

Framework Documentation

- scikit-learn User Guide — https://scikit-learn.org/stable/user_guide.html
- XGBoost Documentation — <https://xgboost.readthedocs.io/>
- SHAP Documentation — <https://shap.readthedocs.io/>
- Streamlit Documentation — <https://docs.streamlit.io/>

APIs

No external third-party APIs are used; the internal SQLAlchemy-based data-access layer is documented within the project repository.

Standards

- ISO/IEC/IEEE 42010:2011 — Systems and software engineering — Architecture description
- OWASP Top 10 — used as a baseline reference for the security design in Section 5.8

Other Technical References

- Gartner. (2023). Gartner forecasts worldwide IT spending to grow 4.3% in 2023. Gartner, Inc.
- Stack Overflow. (2024). Stack Overflow developer survey 2024. Stack Exchange, Inc.

Appendix A: Full Database Schema

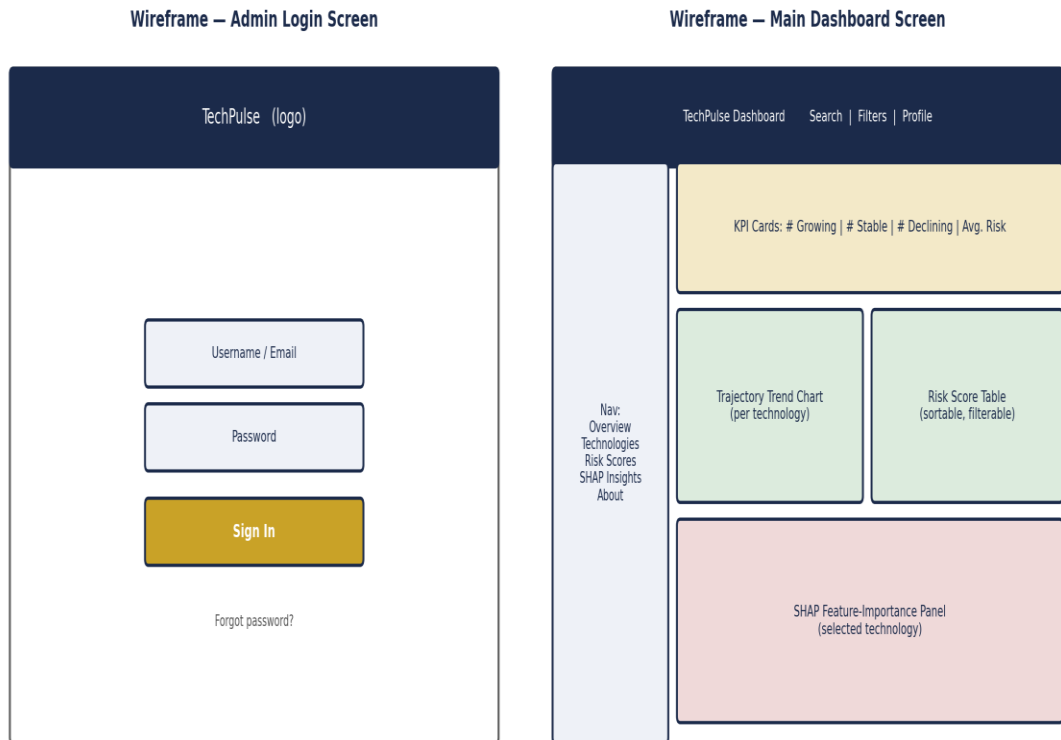
DDL summary of new and reused tables (data types illustrative; full migration scripts to be maintained in the repository's /migrations directory):

```
CREATE TABLE feature_matrix (  
  feat_id    SERIAL PRIMARY KEY,  
  tech_id    INTEGER REFERENCES technologies(tech_id),  
  decay_rate NUMERIC(8,4),  
  growth_momentum NUMERIC(8,4),  
  adoption_breadth INTEGER,  
  sentiment_trend NUMERIC(8,4),  
  computed_at TIMESTAMP DEFAULT now()  
);  
  
CREATE TABLE technology_predictions (  
  pred_id    SERIAL PRIMARY KEY,  
  tech_id    INTEGER REFERENCES technologies(tech_id),  
  model_name VARCHAR(50),  
  trajectory_label VARCHAR(20) CHECK (trajectory_label IN  
    ('growing','stable','declining')),  
  risk_score NUMERIC(5,4),  
  shap_top_features JSONB,  
  predicted_at TIMESTAMP DEFAULT now()  
);
```

Appendix B: Flowcharts

See the pseudocode-based process designs for Modules 1–5 in Section 5.5.1, which double as the textual equivalent of the module flowcharts. Graphical flowcharts will be maintained alongside the source code in the repository's /docs/flowcharts directory as implementation progresses.

Appendix C: Wireframes



Full-size reproduction of Figure 5.4

Appendix D: UML Diagrams

The Level-0 Data Flow Diagram (Figure 5.3) and Entity Relationship Diagram (Figure 5.2) serve as the primary structural and behavioural diagrams for this academic-scope system. A class diagram for the modelling pipeline (DataLoader, FeatureEngineer, ModelTrainer, Explainer, PredictionPublisher classes) will be added during implementation as the module interfaces stabilise.

Appendix E: API Specifications

Internal Function	Description	Returns
get_feature_matrix(tech_id=None)	Fetch engineered features for one or all technologies	DataFrame
get_predictions(filter=None)	Fetch current trajectory predictions and risk scores	DataFrame
get_shap_explanation(tech_id)	Fetch top-N SHAP features for a technology	JSON
trigger_retrain()	Admin-only: triggers the batch retraining pipeline	Job status

Appendix F: Configuration Files

Illustrative environment configuration (values redacted; actual secrets are stored in GitHub Secrets / environment variables, never committed to source control):

```
# .env.example
DATABASE_URL=postgresql://<user>:<password>@<host>:5432/dev_ecosystem_analytics
STREAMLIT_SERVER_PORT=8501
MODEL_STORE_PATH=./artifacts/models
SHAP_CACHE_TTL_SECONDS=3600
LOG_LEVEL=INFO
```